

Project Design Phase
Solution Architecture

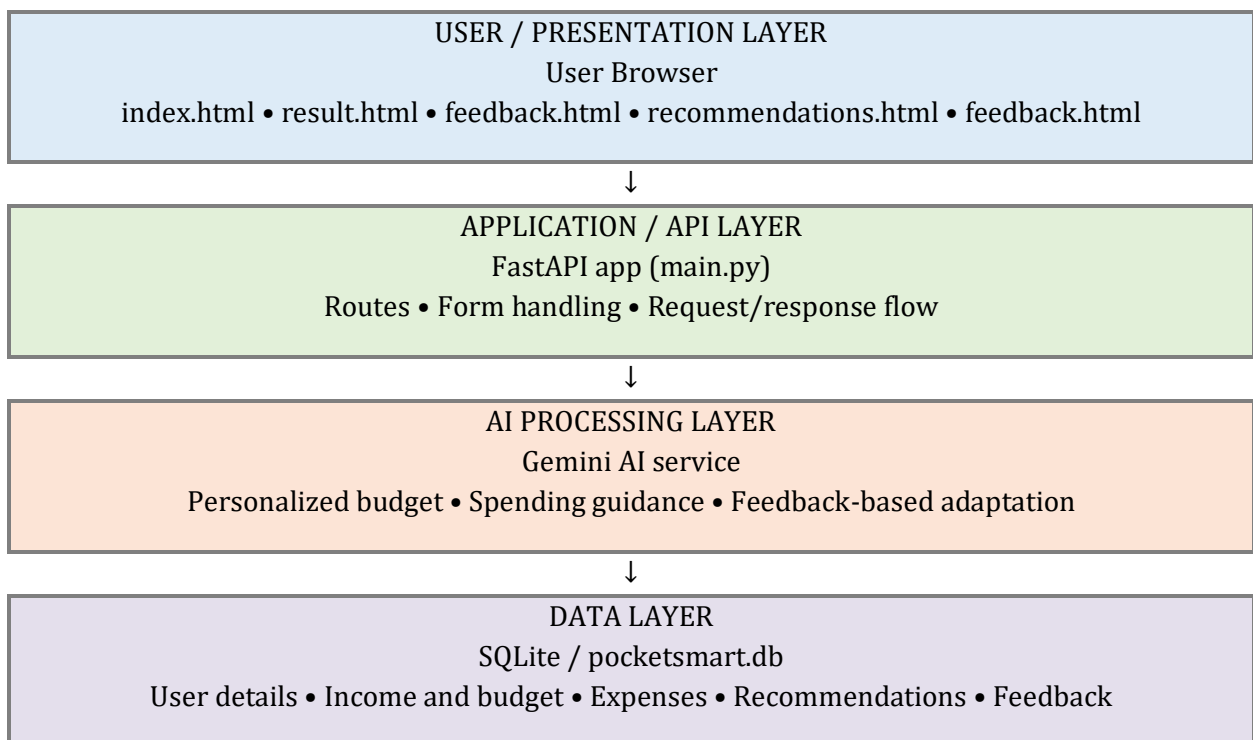
Date	30 September 2026
Team ID	SWTID-2026-3350
Project Name	PocketSmart AI – Your Smart Budget & Recommendation Assistant
Maximum Marks	4 Marks

Solution Architecture:

The PocketSmart AI – Your Smart Budget & Recommendation Assistant solution architecture connects the user's financial requirements with the web interface, FastAPI backend, Gemini AI service, and SQLite database. It defines how user financial information moves through the system, how personalized budget and spending recommendations are generated, how feedback is processed, and how an updated recommendation is returned.

The architecture is divided into presentation, application/API, AI processing, and data layers, with each layer having a clear responsibility.

Figure 1: PocketSmart AI – Your Smart Budget & Recommendation Assistant Architecture and Data Flow



PocketSmart AI – Your Smart Budget & Recommendation Assistant – Architecture Components

Detailed User Data Flow

1. INPUT User enters income, budget, expenses, categories, and spending preferences.	2. REQUEST FastAPI receives the financial form through /analyze-budget.	3. AI Gemini analyzes the budget and generates personalized recommendations.	4. FEEDBACK User submits feedback, satisfaction, affordability, or requested changes.	5. UPDATE The recommendation is adapted and shown on recommendations.html.
---	--	---	--	---

1. Presentation Layer

Jinja2 HTML pages interact with the user. The homepage collects financial profile information, the result page displays budget analysis and spending recommendation content, the feedback page collects the user's experience, and the recommendations page presents the updated recommendation.

2. Application / API Layer

FastAPI handles routing and form submissions. main.py initializes the application and connects routes and static files, while the routes module controls the request and response flow.

3. AI Processing Layer

The AI service uses user financial profile information and budget and spending preferences to generate budget and spending guidance recommendations. Feedback can be used to request a revised plan.

4. Data Layer

SQLite provides local persistence for the prototype. User details, budget summaries, recommendations, and feedback information can be stored as part of the application workflow.

5. Main Modules

Module	Responsibility	Technology
main.py	Starts FastAPI app, mounts static files, includes routes.	FastAPI / Uvicorn
routes.py	Handles homepage, budget analysis, feedback and updated-recommendation flow.	FastAPI / Jinja2
gemini_service.py	Generates AI-assisted budget, spending and recommendation content.	Google Gemini

database.py	Defines database connection and financial/recommendation persistence.	SQLAlchemy / SQLite
HTML templates	Collect and display financial information, budget analysis and recommendations.	Jinja2 / HTML
style.css	Provides visual styling for the web interface.	CSS

PocketSmart AI – Your Smart Budget & Recommendation Assistant – Requirements and Architecture Outcome

6. Functional Requirements

Requirement	Architecture Support
Personalized budget recommendation	Financial input → FastAPI → Gemini → result.html
Spending guidance	AI processing layer returns spending guidance recommendations.
Feedback-based adaptation	feedback.html → /submit-feedback → AI update → recommendations.html
Data persistence	FastAPI/SQLAlchemy stores user financial and recommendation information in SQLite.
Web accessibility	Browser-based Jinja2 interface served by FastAPI.

7. Architecture Behaviour

When the user submits financial profile information, the request reaches FastAPI through the budget analysis route. The application prepares the user financial information for the AI service, receives the generated recommendation, and renders it through the result page. The user can then give feedback about satisfaction, affordability, or requested changes. The feedback is processed to produce an updated recommendation, which is displayed through the recommendations page.

8. Future Expansion

The modular architecture can support future additions such as authentication, financial progress tracking, richer financial and spending data, mobile support, cloud database storage, and integrations with financial or budgeting data integrations.

Architecture Outcome

The architecture supports the complete PocketSmart AI – Your Smart Budget & Recommendation Assistant personalization loop: financial profile → budget analysis → personalized recommendations → feedback → updated recommendation. Separating the interface, API, AI, and data responsibilities makes the system easier to understand, demonstrate, and extend.

Financial Profile → AI Budget Analysis → Recommendations → Feedback → Updated
Recommendation